

multiplicity (“the condition whereby a person displays or experiences multiple distinct personalities or selves in one body”). 11. About Word of the Day 12. Nominate a word 13. Leave feedback <https://en.wiktionary.org/wiki/plurality>

Wikiquote quote of the day:

The only ones left with any confidence at all are the New Dumb. It is the beginning of the end of our world as we knew it. Doom is the operative ethic. --Hunter S. Thompson https://en.wikiquote.org/wiki/Hunter_S._Thompson

FRED TALKS

18th July 2026

CONTENTS

The Training Grounds: A Taxonomy of RL Environments for LLM Agents

HAN, NOT SOLO · 1766 WORDS

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

Ownership

THORSTEN BALL · 1122 WORDS

[Daily article] July 18: Wikipedia:Featured topics/Mercenary War

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 481 WORDS

The Training Grounds: A Taxonomy of RL Environments for LLM Agents

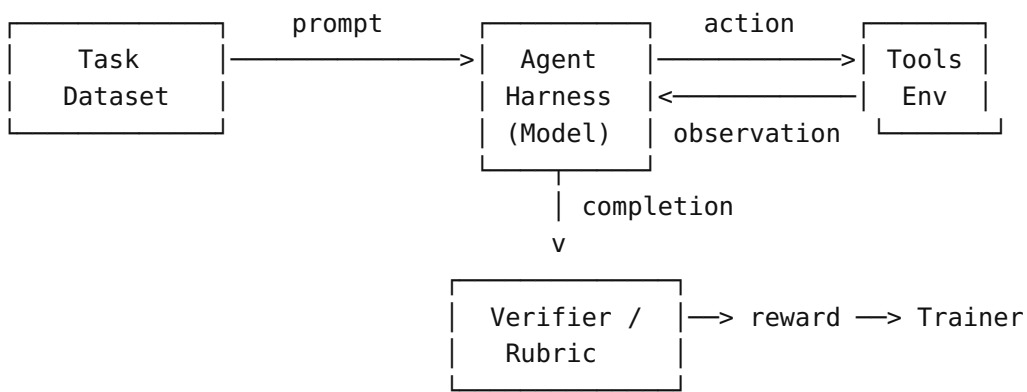
HAN, NOT SOLO · 22 MAR 2026 · [SOURCE](#)

Model architecture gets all the attention. Post-training recipes follow close behind. The training environment — what the model actually practices on, how its work gets judged, what tools it can use — barely enters the conversation. That’s the part that actually determines what the agent can learn to do.

A model trained only on single-turn Q&A will struggle the moment you ask it to maintain state across a 50-step enterprise workflow. A model trained with a poorly designed reward function will learn to game the metric, not solve the problem. The environment isn’t a detail. It’s half the system.

The Canonical Loop

An RL environment for an LLM agent bundles three things: a dataset of task inputs, a harness for the model, and a reward function to score outputs. The training loop looks like this:



Formally, a complete RL environment is a tuple:

Task distribution, harness, verifier, state management, configuration. Let’s go through each.

The task distribution is the set of problems the agent trains on. Not all tasks are equal, and not just in difficulty. They vary structurally in ways that demand different capabilities:

prisoners. From this point, neither side showed any mercy; the unusual ferocity caused contemporaries to term it the "Truceless War". (This article is part of a featured topic: Mercenary War.).

Read more: https://en.wikipedia.org/wiki/Wikipedia:Featured_topics/Mercenary_War

Today's selected anniversaries:

1723:

Johann Sebastian Bach directed the first performance of his cantata Erforsche mich, Gott, und erfahre mein Herz in Leipzig. https://en.wikipedia.org/wiki/Erforsche_mich,_Gott,_und_erfahre_mein_Herz,_BWV_136

1976:

At the Olympic Games in Montreal, Nadia Comăneci became the first gymnast to score a perfect 10 for her routine on the uneven bars. https://en.wikipedia.org/wiki/Nadia_Com%C4%83neci

1989:

American actress Rebecca Schaeffer was shot and killed by Robert John Bardo, eventually prompting the passage of anti-stalking laws in California. https://en.wikipedia.org/wiki/Rebecca_Schaeffer

1995:

Selena's album Dreaming of You, instrumental in popularizing Tejano music, was posthumously released. https://en.wikipedia.org/wiki/Dreaming_of_You_%28Selena_album%29

Wiktionary's word of the day:

plurality: 1. (uncountable) The state of being plural. 2. (ecclesiastical) The holding of multiple benefices. 3. (countable) A state of being numerous. 4. (countable) A number or part of a whole which is greater than any other number or part, but not necessarily a majority. 5. (countable) A number of votes for a single candidate or position which is greater than the number of votes gained by any other single candidate or position voted for, but which is less than a majority of valid votes cast. 6. (countable) A margin by which a number exceeds another number, especially of votes. 7. (countable) A group of many entities: a large number. 8. (countable) A group composed of more than one entity. 9. (of spouses) Polygamy. 10. (psychology) Synonym of

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, ... — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.



You know what *you* should own? A subscription to this newsletter:

[Daily article] July 18: Wikipedia:Featured topics/Mercenary War

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 18 JUL 2026 · [SOURCE](#)

Hamilcar's defeat of Spendius and Autaritus took place in 240 BC during the Mercenary War. The precise location of the battle, in what is now north-west Tunisia, is unknown. A Carthaginian army led by Hamilcar Barca defeated a rebel army led by Spendius and Autaritus. Unable to confront the Carthaginian war elephants and cavalry on open terrain, the rebels stayed on higher and rougher ground and harassed the Carthaginian army, trapping it in a mountain valley. The Carthaginian situation was bleak until the rebel commander Naravas defected, bringing 2,000 cavalry with him, allowing Hamilcar to deploy his army. Spendius chose to engage in a battle but was heavily defeated. Spendius perceived Hamilcar's generous treatment of rebel prisoners as the motivation behind Naravas's defection and had 700 Carthaginian prisoners tortured to death to prevent reoccurrence; the Carthaginians, in turn, killed their

Task Type	What the Agent Must Do	Example Systems
Single-turn Q&A	One prompt → one response, check answer	Math benchmarks, SimpleQA
Multi-hop search	Chain searches, synthesize sources	BrowseComp, WebWalkerQA
Open-ended research	No single correct answer; report quality matters	ADR-Bench, ResearchRubrics
Agentic tool-use	Call tools correctly in sequence	tau-bench, function-calling benchmarks
Stateful enterprise	Modify persistent DB state, work within access controls	EnterpriseOps-Gym
Code execution	Write code, run it, check outputs	SWE-Bench, LiveCodeBench

Training only on tasks with ground-truth answers produces an agent that's never learned to reason under ambiguity. Training only in clean, deterministic environments produces an agent that falls apart in production. The task distribution is a design decision with direct consequences.

Task synthesis is increasingly a first-class problem. With real-world research tasks, you rarely have a large labeled dataset. Strategies that have emerged:

- **Back translation:** Start from a desired output, reconstruct the query that would produce it
- **Graph-based synthesis:** Build a knowledge graph, generate multi-hop queries over it
- **Automated environment generation:** Use LLM coding agents to write new environment code. [AutoEnv](#) reports ~\$4/env average cost.
- **Curriculum construction:** Order tasks by difficulty and increase complexity during training

The cheapest-to-collect tasks are single-turn with verifiable answers. The most valuable tasks for long-horizon behavior are expensive to construct. This tension drives most environment design decisions.

The harness is the scaffolding that mediates between the model and the environment. It controls how the model interacts, not what it knows.

```
H = {
    rollout_protocol, # SingleTurn | MultiTurn | Agentic
    tools,            # Available tools in this episode
    system_prompt,    # Instructions for the agent
    context_manager,  # How to handle context overflow
```

```

    turn_limit,      # Max interactions per episode
    sandbox,         # Code execution sandbox
    state            # Persistent state across turns
}

```

Rollout protocols range from trivial to complex:

Harness Type	Description	When to Use
Single-Turn	One prompt, one response	Math, factual QA
Multi-Turn	Back-and-forth dialogue	Games, structured tasks
Tool-Use	Model calls tools, receives results	Agent benchmarks
Stateful Tool-Use	Tools modify persistent state	Enterprise workflows, SWE-Bench
Agentic ReAct	Full Thought → Action → Observation loop	Deep research, complex workflows

Tools span a wide taxonomy:

Category	Tools	Deterministic?	Stateful?
Information retrieval	web_search, scholar_search	No (live web)	No
Content extraction	jina_reader, visit, web_scrape	No	No
Code execution	python_interpreter, shell, sandbox	Yes (given same code)	Yes
File operations	file_read, file_write	Yes	Yes
Browser automation	playwright, link_click	No	Yes
Task management	todo, section_write	Yes	Yes

The mix of deterministic/non-deterministic and stateful/stateless tools has real implications for reproducibility and reward assignment. Non-deterministic tools mean two runs of the same trajectory can produce different outcomes — which complicates both debugging and verifier design.

Context management is where most teams underinvest, especially for long-horizon tasks. A 600-turn research episode blows past any practical context window. Strategies used in production:

Strategy	Description	Trade-off
Recency-based retention	Keep N most recent turns	Simple, but loses early context
Markovian reconstruction		Principled, expensive

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it’s actually deployed?

- If you think your colleagues needs to know about this change, because it’s new feature they should all test, or it’s a new convention in codebase, or maybe it’s a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.

- Do customers need to know? Who reported the bug? Who’s blocked? Let them know.

- Does the world need to know? Announce that it’s out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that’s a lot. And there’s actually more, because I’m sure I forgot some stuff.

But that’s how you build a product in a small team. We don’t have PMs, we don’t have a Q&A department. We’re small, but we’re *great*, we can do all of that.

And it’s always okay to ask for help, it’s okay to ask questions, it’s okay to redo things and triple-check. What’s not okay is to implicitly assume that someone else will do the things here that you haven’t thought about.



“How does this apply juniors? You can’t expect them to really do all of that?”

I’ve been asked these questions, or variations of them, after I shared the thoughts above and here’s my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

If I ask you “can you own this?” or “can you take care of this?” or “are you on it?” — what I’m doing is I’m asking you to *own* it, to own the solution of a problem from end to end. From “we have a problem” to “we don’t have to think about it again.”

That means, when you say that you’re owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we’re actually trying to solve here. Maybe you think “the problem is that we need to migrate from using X to using Y”, but that’s not a problem, that’s a solution. The problem is likely something like “performance is bad”, “it’s not stable”, “it fails for customer x”. Maybe there’s other possible solutions to that? Think about those. What are the tradeoffs? What’s the best solution to go with considering these tradeoffs?
- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?
- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?
- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven’t confirmed yet?
- Think about how you’d test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?
- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!
- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say “here’s what I built, under these constraints, with these tradeoffs?”
- Test it manually. Yes, there’s automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

	Reconstruct state from scratch each turn	
Reference-preserving summarization	Summarize old context, keep citations	Preserves verifiability
Reference-preserving folding	Compress context without losing references	Best for research tasks

An agent doing multi-hour research needs to remember why it started searching in a particular direction twelve tool calls ago. Dropping that context causes repeated work and lost threads.

The verifier maps a completion to a reward:

In Atari, the score is unambiguous. In deep research, what counts as a good answer is not. The verifier is where this gets hard.

Type	Reward Signal	When to Use
Exact match	Binary (0/1)	Ground truth available
Code execution	Binary or partial	Output can be tested programmatically
LLM-as-judge	Continuous [0,1]	Open-ended quality, no other option
Checklist-style	Continuous	Multi-criteria research tasks
Evolving rubric (RLER)	Continuous	Resistant to reward hacking
Process reward model (PRM)	Per-step continuous	Long-horizon credit assignment
Pairwise comparison	Relative rank	Relative quality matters more than absolute
Multi-criteria composite	Weighted sum	Multiple quality dimensions

A few principles that actually matter in practice:

Verifiable beats judgeable. Programmatic checks — code execution, string match — are faster, cheaper, and more consistent than LLM-as-judge. Use LLM-as-judge when there’s no other option, not as the default.

Reward granularity is a separate decision from reward type. You can score at the trajectory level (did the final output pass?), turn level (was each tool invocation useful?), or per-step with process rewards. Turn-level supervision, as Nanbeige4.1 does across up to 600 tool calls, enables finer credit assignment — the model can learn that the problem was a bad search query in turn 23, not that the entire episode failed.

Static rubrics get gamed. Models learn to write answers that score well on your rubric rather than solving the problem. DR Tulu’s RLER (Rubric-Level Evolving Reward) co-evolves the rubric with the policy during training. Harder to exploit a moving target.

Noise injection is underrated. Step-DeepResearch deliberately injects 5–10% tool errors during training. The resulting model handles flaky APIs and unexpected failures in production significantly better.

State management determines whether the environment is stateless or persistent. Most academic benchmarks are stateless — each episode starts fresh. Enterprise environments are not. [EnterpriseOps-Gym](#) maintains 164 database tables and 512 tools across episodes. Actions in one task affect the state seen by subsequent tasks. That’s a fundamentally different problem for agents to solve.

Configuration covers turn limits, context budgets, sampling temperature, and curriculum scheduling. These are not afterthoughts. A turn limit of 5 vs. 600 changes what skills the agent can develop. [AgentScaler](#) uses a two-phase curriculum — fundamental capabilities first, then domain-specific tasks — and the ordering matters. Step-DeepResearch progressively scales context windows from 32K to 128K during mid-training.

The most consequential architectural question: where does the model sit relative to the environment?

Option A: Model outside the environment (decoupled). Model is served via API; the environment calls it at each step. Clean separation. Easy to swap models.

Option B: Model inside the environment (co-located). Model and environment share the same training loop. Lower latency, tighter integration, harder to reuse.

Option C: Split architecture. Trainer, model inference server, and environment are three separate processes communicating via API. This is where the field is landing.

How real systems implement this:

System	Topology	Notes
Prime Intellect verifiers	Option C (Split)	Env is a standalone Python package distributed via Environments Hub
Tongyi DeepResearch	Option B (Co-located)	Tools, context manager, verifiers inside the training pipeline Single-agent ReAct loop embedded in training

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

Ownership

THORSTEN BALL · 08 JUL 2026 · [SOURCE](#)

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It’s only lightly edited.

I shared it before, but not here, because I didn’t think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don’t I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.



Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase “ownership” or allude to it, but haven’t explained what “ownership” means to me in a while.

So, ownership.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our [data plugin](#) for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: [Extending Claude’s capabilities with skills and MCP servers](#)

Distribute skills from an MCP server

It’s increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. [Canva](#), [Notion](#), [Sentry](#), and more do this today in Claude, listing the skill next to their connector in our [web directory](#).

To make that pairing portable across every client, the MCP community is actively working on an [extension](#) for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it’s the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Step-DeepResearch	Option B (Co-located)	
MiroThinker	Option C (Split)	Tool servers and sandbox run independently from model
Tinker API	Option A (Decoupled, cloud)	Model stays remote; researcher sends <code>forward_backward</code> + <code>sample</code> calls via API
AutoEnv	Option A (Decoupled)	CoreEnv/ObsEnv/SkinEnv abstraction layers
EnterpriseOps-Gym	Option A (Decoupled)	Containerized sandbox accessible via any model API

The trade-offs:

Dimension	Model Outside (A/C)	Model Inside (B)
Flexibility	Swap models easily; env is reusable	Tighter integration
Scalability	Scale inference and training independently	Must scale everything together
Portability	Env packages are shareable	Env tied to training framework
Latency	Network overhead per tool call	No network overhead
RL compatibility	Works with any RL trainer	Usually tied to one trainer

The field has converged on Option C for production training. Prime Intellect’s architecture — environments as standalone installable packages that communicate with models via OpenAI-compatible API endpoints — is becoming the standard. The payoff: environments are publishable, trainer-agnostic, and inference and environment execution can be parallelized across nodes.

Tinker pushes this further by making even the training compute remote. The researcher controls the algorithm and never touches model weights. The environment’s job is purely generating experience.

Practical Decision Framework

When building an RL environment for an LLM agent:

Do you have ground truth answers?

- Yes → Exact-match or code-execution verifier
- No → LLM-as-judge, checklist, or pairwise comparison

How many tool calls per episode?

- < 5 → Single-turn or simple tool-use env, no context management needed
- 5–50 → Multi-turn with basic context management
- 50–600 → Full agentic env with reference-preserving context management

Where should the model live?

- Experimenting across many environments → Model outside (Option A/C), use Prime Intellect Hub
- Tight RL training loop → Model co-located (Option B)
- No GPU access → Tinker API

What reward granularity?

- Simple tasks → Outcome-level
- Long-horizon tasks → Turn-level (Nanbeige4.1) or process rewards (PRIME)
- Open-ended research → Evolving rubrics (RLER) or checklist-style (Step-DR)

How to scale environments?

- Manual curation → High quality, expensive, this is where you start
- [AutoEnv](#) → Automated generation at ~\$4/env
- [AgentScaler](#) → Systematic scaling of heterogeneous simulated environments

Three things to pay attention to.

Environment diversity matters as much as environment quality. [AgentScaler](#)'s key finding is that heterogeneity of environments drives capability breadth in ways that simply adding more data from the same distribution cannot. You need more kinds of environments, not just more environments.

Making MCP clients more context-efficient

MCP standardizes how AI agents (*[clients](#)*) connect to and work with tools and data sources they need (*[servers](#)*). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

[Tool search](#) defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our [testing](#), tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: [advanced tool use](#)

Process tool results in code with programmatic tool calling

[Programmatic tool calling](#) processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly [37%](#) on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See [advanced tool use](#) for the full breakdown.

Pairing MCP servers with skills

[Skills and MCP are complementary](#). MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

[Plugins](#) for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

[Elicitation](#) lets your server pause mid-tool call to ask the user for input. [Form mode](#) sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. [URL mode](#) hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest [MCP spec](#) supports [CIMD](#) (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. [Vaults](#) in [Claude Managed Agents](#) covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Automated environment generation is viable. At \$4 per generated environment, cost is no longer the bottleneck. The bottleneck is verifier quality — auto-generated environments with weak reward functions will teach the wrong behaviors at scale. ([AutoEnv](#))

The environment-as-package model is winning. The [Prime Intellect Environments Hub](#) is creating a shared ecosystem around RL environments, in the same way PyPI and HuggingFace created ecosystems around code and model weights. Environments published once, consumed by any trainer. This is a significant infrastructure shift.

The model isn't the only variable. The training ground shapes what the model can become. The task distribution, the harness, the verifier, the state management, the topology — these are the decisions that separate agents that work from agents that demo.

References

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {The Training Grounds: A Taxonomy of RL Environments for},
  year = {2026},
  month = {03},
  day = {21},
  howpublished = {\url{https://leehanchung.github.io}},
  url = {https://leehanchung.github.io/blogs/2026/03/21/rl-environm
}
```

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the M×N integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.